JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
M.Sc. Sem-2
Data Science and Artificial Intelligence (DSAI)/
Cyber Security (C.S.)

EXPERIMENT NO:11 **DATE: / /**

AIM : SERVO MOTOR (SG-90) CONTROL USING ESP32

OBJECTIVE: The main objective of this experiment is,

A. Servo motor (SG-90) control using ESP32.

APPRATUS: ESP32 Board, Servo Motor (SG-90), Jumper Wire, Breadboard

DESCRIPTION:

Servo motors are geared DC motors that have an integrated servomechanism with a feedback loop to allow precise positioning of the motor shaft. A high gear ratio allows a small servo to have an impressive torque rating.

Most servos are limited in rotation to either 180 or 270 degrees, with 180-degree servo motors being more common. There are specially modified servo motors that can rotate beyond 360-degrees, but we won't be working with those today.

Servo motors come in a wide range of sizes and can be controlled either with an analog PWM signal or with a digital I/O signal.

A very common and inexpensive servo motor is the SG90. It's a small plastic-gear servo that has become a standard for experimenting and is also used in a lot of RC hobby applications. It has a "cousin", the MG90, which has similar specifications but uses metal gears.

The SG90, like most servo motors, can be powered from 5-6 volts, so servo motors are great for battery-powered applications.

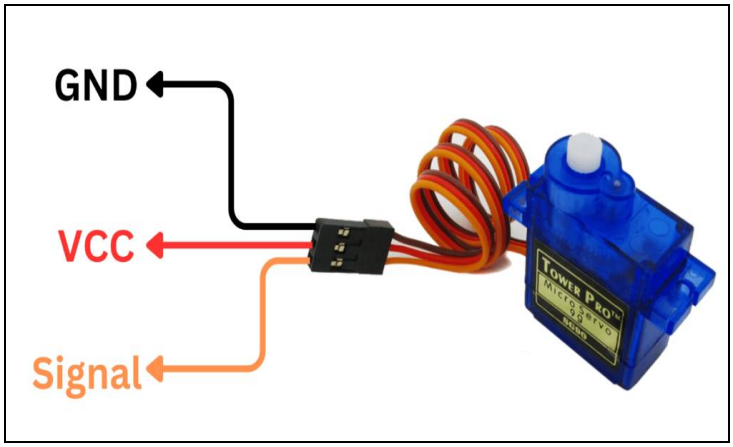
A standard hobby servo contains four essential parts working in unison:

DC Motor: The actual drive unit that provides rotation.

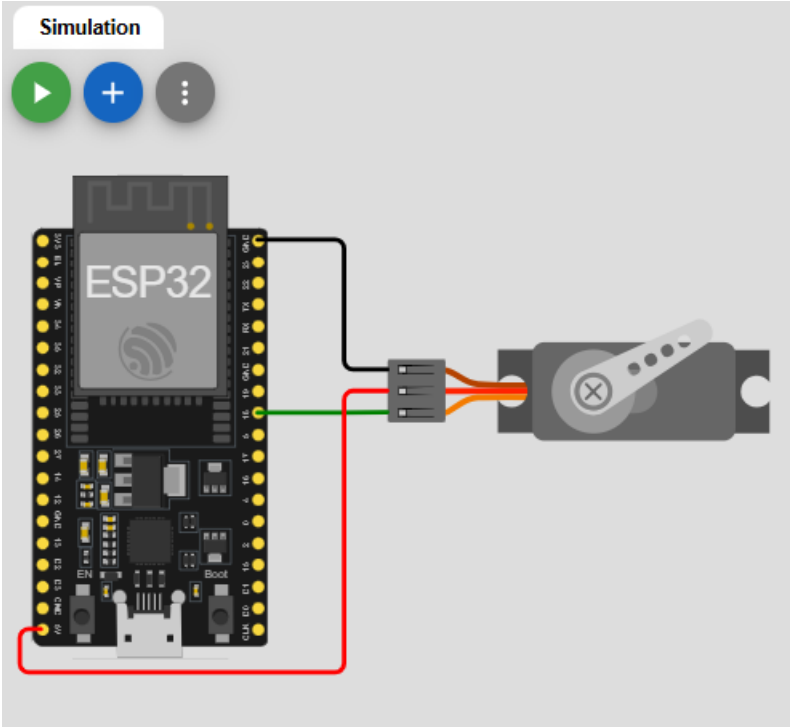
Gearbox: A set of gears (plastic in SG-90) that reduces the motor's high speed to increase torque (lifting power).

Potentiometer (Position Sensor): A variable resistor connected to the output shaft that provides a voltage signal representing the shaft's actual position.

Control Circuit: The "brain" that receives commands and manages the feedback loop.



INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- **Start:** Begin program execution (when the ESP32 boots up).
- **Define Pin:** Specify which **GPIO pin** (e.g., GPIO 18) will send the control signal to the servo motor.
- **Initialize Serial (Optional):** Start serial communication for debugging or user input.
- **Configure PWM Frequency:** Set the ESP32's internal PWM generator frequency to **50 Hz** (a 20ms cycle) as required by standard hobby servos.
- **Attach Servo Object:** Link a software "servo object" to the specified GPIO pin and define the minimum and maximum pulse widths in microseconds (e.g., **500 µs** for 0° and **2400 µs** for 180°).
- **Determine Target Angle:** Decide the next desired position (e.g., use a `for` loop to sweep from 0 to 180).
- **Calculate Pulse Width:** Convert the target angle (0–180) into the precise corresponding pulse width (e.g., 1.5ms for 90°). The library handles this conversion.
- **Send Signal:** Issue the command to the hardware to generate a pulse of the calculated width on the specified GPIO pin.
- **Pause:** Introduce a small delay (e.g., 15 ms) to allow the motor time to physically move to the new position smoothly before receiving the next command.
- **Repeat:** Loop back to Step 1 to potentially determine a new target angle or simply maintain the current one.

PROGRAM:

RESULTS:

CONCLUSION:

SAMPLE VIVA-VOCE QUESTIONS:

1) What is a servo motor?
2) How does a servo motor work?
3) How the SG-90 servo is motor connected to the ESP32?
4) How can we control the position of the servo motor using the ESP32?
5) What is the range of motion for the SG-90 servo motor?